

Containers e Docker

Criação, gerenciamento e deploy de aplicações

- Data de conclusão: 29 de maio de 2026 às 23:59
- Objetivo: padronizar ambientes e reduzir variação operacional
- Resultado: deploy previsível, auditável e reversível

Data de conclusão: 29 de maio de 2026 às 23:59

Docker não é só uma ferramenta; é um padrão de entrega.

Agenda

- O que é container
- Imagem e container na prática
- Fluxo de criação e execução
- Instalação no Windows e Linux
- Comandos de rotina
- Estratégia de deploy em produção

1
Etapa

2
Etapa

3
Etapa

4
Etapa

5
Etapa

6
Etapa

1) O que é um container?

- É uma aplicação empacotada com runtime e dependências.
- A execução fica isolada do host e de outros serviços.
- Evita o clássico problema “na minha máquina funciona”.
- Mantém comportamento consistente entre homologação e produção.

Ritmo operacional

Resumo

- Padronização
- Consistência
- Reprodução exata

2) Imagem x Container

- Imagem é o artefato: pacote imutável e versionado.
- Container é a instância em execução da imagem.
- A cada build, nasce uma imagem nova com camadas novas.
- Rollback é retorno rápido para uma tag anterior estável.

Implementação

Resultado

- Imutabilidade
- Governança de versão
- Recuperação rápida

Checklist

- A cada build, nasce uma imagem nova com camadas novas.
- Rollback é retorno rápido para uma tag anterior estável.

3) Como funciona o fluxo

- Criar Dockerfile com contexto e entrypoint.
- Construir imagem: docker build.
- Executar em teste local com portas e variáveis.
- Publicar em registry com tag imutável.
- Implantar por ambiente com controle de mudança.

Ritmo operacional

Resumo

- Build > Test > Tag > Release
- Reprodutibilidade no ciclo

4) Comandos essenciais

01. `docker build -t projeto:1.0.0 .`

02. `docker images`

03. `docker run -d --name projeto -p 3000:3000 projeto:1.0.0`

04. `docker ps`

05. `docker stop projeto`

06. `docker logs projeto`

07. `docker exec -it projeto /bin/sh`

Observação de operação

Notas

- Essenciais para implantação diária
- Use ``docker ps`` e ``logs`` no monitoramento

5) Instalação no Windows

- Instalar Docker Desktop.
- Habilitar WSL 2.
- Validar com `docker --version`.
- Executar `docker run hello-world`.
- Ajustar CPU e memória conforme necessidade do cenário.

Comando de validação

```
docker --version
```

Notas

- Validação curta e objetiva
- Configuração inicial segura

Check de pré-go-live

6) Instalação no Linux

- Atualizar pacotes e instalar docker-ce.
- Ativar serviço do Docker com `systemctl enable --now docker`.
- Adicionar usuário ao grupo docker.
- Reautenticar sessão e validar com hello-world.
- Configurar limites de recursos por host.

Comando de validação

```
systemctl enable --now docker
```

Notas

- Padrão server-friendly
- Base sólida para automação

Check de pré-go-live

7) Estrutura de deploy

- Tags semânticas para produção (sem usar latest).
- Secrets e variáveis fora da imagem.
- Health checks e política de restart.
- Observabilidade por logs, CPU e memória.
- Plano de rollback documentado por versão.

Ritmo operacional

Resumo

- Confiabilidade
- Segurança operacional
- Recuperação garantida

Encerramento

- Contêineres tornam o ambiente previsível e repetível.
- O deploy ganha controle por imagem e por versão.
- Com isso, equipes liberam mais rápido com menos risco.

Menos surpresa operacional, mais confiança na entrega.

Próximo passo: primeiro container de produção em 30 minutos